Untersuchung und Report

Views

Welchen Zweck haben Views?

- Views vereinfachen komplexe Abfragen und machen die Tabellen übersichtlicher.
- Die Abfragen werden als virtuelle Tabellen gespeichert und zeigen so die Komplexität der darunterliegenden Datenstrukturen nicht mehr.
- Praktisch, um die Daten genau an die jeweiligen Bedürfnisse anzupassen.

Welche Vorteile bieten Views gegenüber dem direkten Zugriff auf Tabellen?

- Mit Views spart man sich Aliase und Joins. Außerdem muss man sich nicht mehr darum kümmern, in welchen Tabellen genau welche Spalten im Hintergrund liegen.
- Man kann nur bestimmte Spalten freigeben, z. B. zeigt `kunde_informationen` nur Kundennummer, Name und E-Mail. Geburtsdatum, Telefon und Adresse bleiben versteckt (Datenschutz).
- Häufig gebrauchte Abfragen muss man nur einmal schreiben und kann sie dann immer wieder verwenden.
- VIEW besteht auch nach Schließen der aktuellen Sitzung weiter und wird erst durch DROP VIEW gelöscht. (TEMPORARY VIEW existiert nur während der aktuellen Sitzung.)

Welche Nachteile bzw. Gefahren können entstehen?

- Performance, da die View keine echten Daten speichert, sondern bei jeder Abfrage die SQL-Abfrage im Hintergrund neu ausführt.
- Man sieht von außen nicht, wie aufwendig die Abfrage im Hintergrund ist.
- Read-only (kein INSERT, UPDATE oder DELETE möglich).
- Keine Indizes möglich.
- View kann nicht geändert werden, sondern muss mit DROP VIEW gelöscht und dann neu erstellt werden.

Performance

Liefern die beiden Views tatsächlich dasselbe Ergebnis?

Ja, laut Angabe sind die beiden Abfragen semantisch identisch und liefern beide die letzte(n) Bestellung(en) pro Kunde. Überprüfen könnte man das mit `EXCEPT` in beide Richtungen. Das habe ich aus Zeitgründen nicht mehr gemacht, weil Variante A sehr lange braucht.

Wie unterscheiden sich die gemessenen Laufzeiten?

Gemessen mit `timeit`, 10 Wiederholungen, Abfrage `SELECT * FROM ... WHERE kunde_id = 42 LIMIT 10000;`

```
Mit Limit in views.sql:
Ergebnis A für 1 Ausführung: 3 Minuten 55.350 Sekunden
Ergebnis B für 1 Ausführung: 0 Minuten 0.044 Sekunden
Faktor: 5381.697
Gesamtdauer: 39 Minuten 13.938 Sekunden


Mit Limit in benchmark.py:
Ergebnis A für 1 Ausführung: 3 Minuten 59.527 Sekunden
Ergebnis B für 1 Ausführung: 0 Minuten 0.045 Sekunden
Faktor: 5327.925
Gesamtdauer: 39 Minuten 55.723 Sekunden
```

Die View mit dem Subselect (A) ist ca. 5.400-mal langsamer als die View mit dem Join (B), obwohl beide von außen (SELECT in benchmark.py) gleich abgefragt werden.

Warum ist es problematisch, dass die interne SQL-Abfrage einer View bei der Verwendung nicht unmittelbar sichtbar ist?

Weil man schlecht abschätzen kann, wie performant die Abfrage dahinter ist. Zwei Views können gleich aussehen und dasselbe liefern, aber völlig unterschiedlich lange brauchen. Das merkt man erst beim Messen oder mit EXPLAIN QUERY PLAN.

Was zeigt `EXPLAIN QUERY PLAN` über die beiden Abfragen?

Grundsätzlich zeigt EXPLAIN QUERY PLAN schrittweise, wie die SQL-Abfrage verarbeitet und ausgeführt wird, ohne die Daten der Abfrage anzuzeigen.

```
--> Limit in views.sql und benchmark.py:

sqlite> EXPLAIN QUERY PLAN SELECT * FROM letzte_bestellung_a WHERE kunde_id
= 42;
QUERY PLAN
|--CO-ROUTINE letzte_bestellung_a
|  |--SCAN b
|  `--CORRELATED SCALAR SUBQUERY 3
|     `--SEARCH b2
`--SCAN letzte_bestellung_a

sqlite> EXPLAIN QUERY PLAN SELECT * FROM letzte_bestellung_b WHERE kunde_id
= 42;
QUERY PLAN
|--CO-ROUTINE letzte_bestellung_b
|  |--CO-ROUTINE letzte
|  |  |--SCAN bestellung
|  |  `--USE TEMP B-TREE FOR GROUP BY
|  |--SCAN b
|  |--BLOOM FILTER ON letzte (kunde_id=?)
|  `--SEARCH letzte USING AUTOMATIC COVERING INDEX (kunde_id=?)
```

```
`--SCAN letzte_bestellung_b

--> Limit in benchmark.py:
sqlite> EXPLAIN QUERY PLAN SELECT * FROM letzte_bestellung_a WHERE kunde_id
= 42 LIMIT 10000;
QUERY PLAN
|--SCAN b
`--CORRELATED SCALAR SUBQUERY 3
   `--SEARCH b2

sqlite> EXPLAIN QUERY PLAN SELECT * FROM letzte_bestellung_b WHERE kunde_id
= 42 LIMIT 10000;
QUERY PLAN
|--CO-ROUTINE letzte
|  `--SCAN bestellung
|--SCAN b
|--BLOOM FILTER ON letzte (kunde_id=?)
`--SEARCH letzte USING AUTOMATIC PARTIAL COVERING INDEX (kunde_id=?)
```

- SCAN: geht die ganze Tabelle durch.
- CORRELATED SCALAR SUBQUERY: das Subselect hängt von der äußeren Zeile ab und wird für jede Zeile neu ausgeführt.
- USE TEMP B-TREE FOR GROUP BY: legt einen temporären B-Baum (Suchbaum) für das schnellere Gruppieren an.
- AUTOMATIC COVERING INDEX: SQLite baut sich selbst einen temporären Index, um beim Join schnell nachschauen zu können.
- SEARCH: sucht gezielt, statt alles durchzugehen.
- CO-ROUTINE: die View wird zuerst für sich berechnet und erst danach wird gefiltert.

Welche Unterschiede in der Verarbeitung erklären die gemessenen Laufzeiten?

- Bei A wird für jede einzelne Bestellung das Subselect mit MAX() neu ausgeführt. Bei 100.000 Bestellungen ist das sehr viel Arbeit.
- Bei B wird das MAX() pro Kunde nur einmal mit GROUP BY berechnet, dann ein Index gebaut und danach nur noch schnell nachgeschaut.
- Ich habe zusätzlich ohne LIMIT in der View gemessen (A: 3 Minuten 59.527 Sekunden, B: 0.045 Sekunden, Faktor 5327.925). Die Laufzeiten sind fast gleich wie mit LIMIT in der View. Der große Unterschied kommt also vom korrelierten Subselect in A und nicht vom LIMIT.
- Ohne LIMIT in der View verschwindet im Query Plan das CO-ROUTINE der View. Das korrelierte Subselect bei A bleibt aber, deshalb ist A trotzdem gleich langsam.

Schlussfolgerung

> **Kann eine View eine Abfrage vereinfachen, ohne dass dadurch die zugrunde liegende Komplexität und deren mögliche Auswirkungen auf die Performance verschwinden? Begründe deine Antwort anhand deiner Messungen.**

Ja. Die View macht die Abfrage für den Benutzer einfacher, die Komplexität im Hintergrund bleibt aber. Beide Views werden mit genau derselben Abfrage verwendet, trotzdem ist A ca. 5.400-mal langsamer als

B. Von außen sieht man das nicht, erst durch Messen und EXPLAIN QUERY PLAN.

- Anfangs habe ich übersehen, dass ich sowohl in views.sql als auch in benchmark.py auf 10.000 limitiert habe. Eine zweite Messung ohne LIMIT in der View hat aber fast dieselben Zeiten ergeben. Der Unterschied liegt also wirklich in der Art, wie die Abfrage in der View geschrieben ist (Subselect vs. Join). Damit zeigt sich das Problem, dass man durch die View zwar einfachere Selects schreiben kann, aber übersehen werden kann was genau in der View steht.